

PROJECT SYNOPSIS

Agentic RAG-Based Citizen Service Advisory Chatbot

GitHub Repository: github.com/abhi6744/Agentic-RAG-Based-Citizen-Service-Advisory-Chatbot

Submitted by Abhinav G

Roll No: 25225040

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



School of Sciences

2026-2027

1.1 Introduction

Government citizen service delivery in India increasingly relies on physical service centers such as Akshaya centers in Kerala, which act as front-end delivery points for a wide range of government-to-citizen (G2C) services including certificates, ration card services, Aadhaar-related support, and welfare scheme applications. While these centers bridge the digital divide effectively, the information layer behind them is still largely manual: operators depend on memory or scattered documents to explain eligibility, required documents, fees, and procedures to citizens. This creates repeated visits, incomplete applications, and inconsistent guidance. This project proposes a Citation-Grounded Multilingual Retrieval-Augmented Generation (RAG) Assistant that retrieves information strictly from verified government sources and generates grounded, cited answers in both English and Malayalam, closing the gap between citizens' questions and accurate, trustworthy answers.

1.2 Alignment with UN Sustainable Development Goals (SDGs)

The project aligns primarily with **SDG 10 — Reduced Inequalities**, since it improves equitable access to public services for citizens with limited digital literacy or English proficiency by providing information in their own language and in simple terms. It carries a strong secondary alignment with **SDG 16 — Peace, Justice and Strong Institutions**, because it improves transparency, consistency, and accountability of public-service information delivery through verifiable, citation-grounded answers rather than informal or memory-based guidance.

SDG	Alignment
SDG 10 – Reduced Inequalities	Improves access to public services for citizens regardless of language or digital literacy
SDG 16 – Peace, Justice and Strong Institutions	Improves transparency and consistency of information delivered by public institutions

1.3 Objectives

1. Design a RAG pipeline that retrieves relevant chunks from official Akshaya/government documents before generating any answer.
2. Implement multilingual query handling supporting English and Malayalam. (if LLM limit is available).
3. Build a citation-mapping module so every generated answer is traceable to its exact source document.
4. Implement a confidence-scoring mechanism that flags low-confidence answers and warns the user instead of guessing.
5. Provide eligibility, required-document, fee, and step-by-step guidance for a defined set of high-demand services.
6. Build an admin interface for document ingestion, update, and knowledge-base management.
7. Evaluate the system quantitatively on retrieval relevance, citation accuracy, and response latency against a manual/no-RAG baseline.

1.4 Existing System and Its Limitations

Currently, citizens rely on manual explanation by Akshaya operators or self-navigation of scattered government websites and PDFs. This approach has the following limitations in practice:

- **No standardized answers:** Different operators may give inconsistent explanations for the same service.
- **No source verification:** Citizens have no way to confirm whether an explanation is current or accurate.
- **Language barrier:** Many official documents exist only in complex English or formal Malayalam, unreadable for average citizens.
- **No confidence signaling:** There is no way to indicate when an operator or system is unsure of an answer.

- **Repeated visits:** Missing information about documents/fees leads to citizens visiting centers multiple times.
- **No searchable knowledge layer:** Existing government portals are not optimized for natural-language question answering.

1.5 Proposed System

The proposed system is a middleware-style RAG assistant that sits between the citizen/operator and the pool of official documents. It is structured around four cooperating subsystems:

- **Retrieval Engine** - converts citizen queries into embeddings and searches a vector database of chunked official documents to find the most relevant evidence.
- **Generation Engine** - uses an LLM to compose a natural-language answer strictly from the retrieved evidence, avoiding fabricated claims.
- **Citation & Confidence Module** - attaches source citations to every claim and computes a confidence score based on retrieval similarity, flagging weak matches.
- **Admin & Knowledge Management Module** - allows an administrator to ingest, update, and version official documents that feed the knowledge base.

Unlike a generic chatbot, this system never answers purely from the LLM's internal memory; every claim must be grounded in retrieved, verifiable source text.

1.6 Specialization Concepts Applied

Concept	Applied As
Information Retrieval	Vector similarity search over chunked official documents
Natural Language Generation	LLM-based grounded answer synthesis
Multilingual NLP	Query/answer handling in English and Malayalam
Trust & Verification Design	Citation mapping and confidence scoring
Knowledge Base Management	Document ingestion, chunking, versioning
Human-Centered Design	Operator dashboard and citizen-facing simple UI

1.7 Expected Outcomes

- A working prototype demonstrating multilingual, citation-grounded question answering for at least 4-5 Akshaya services.
- Quantified improvement in answer relevance and citation accuracy compared to a non-RAG LLM baseline.
- A reusable knowledge-ingestion pipeline that can scale to additional services and languages.
- Complete technical documentation (Synopsis, SRS, Design Document) and a public repository for reproducibility.

1.8 Tools and Technologies (Proposed)

Layer	Technology
Backend	Python 3.11+, FastAPI
Embeddings	Sentence-Transformers / multilingual embedding model

Vector Database	FAISS or ChromaDB or Qdrant
LLM Integration	Anthropic/OpenAI API (pluggable provider interface)
Metadata Storage	SQLite / PostgreSQL
Frontend	React or Streamlit or flask
Testing	Pytest, manual evaluation scripts

